

Open Ended Lab



Spring 2026

Digital System Design Lab

Submitted by:

Syed Hamza(23PWCSE2237)
Muhammad Jalal Khan (23pwcse2262)

Class Section : **A**

Date: 7 July 2026

Submitted to: Engr. Habibullah

Department of Computer Systems Engineering
University of Engineering and Technology, Peshawar

4.1 Objective

This project designs and implements a simplified Breakout (brick-breaker) game entirely in FPGA hardware, using Verilog HDL with no microprocessor, soft-core CPU, or software execution of any kind. It solves the digital-system-design problem of generating a real-time, flicker-free VGA video signal and a fully hardware-resident game state machine — paddle position, ball position/velocity, and a 5×10 brick array — that update synchronously with the VGA frame timing. The result is a self-contained, deterministic hardware game controller that drives paddle motion from push-button input and renders paddle, ball, bricks, and border directly to a VGA monitor.

4.2 System Overview

The system is partitioned into four Verilog modules instantiated under a single top module (top.v): a clock divider, a debounce/synchronizer, a VGA sync generator (vga_sync.v), and the game logic core (game_core.v). The board's 100 MHz onboard oscillator is divided by 4 using a free-running 2-bit counter to produce a 25 MHz pixel clock, matching the standard $640 \times 480 @ 60\text{Hz}$ VGA pixel rate. The two push buttons and the active-low reset are first inverted to active-high and then passed through three instances of debounce_sync, which synchronize each signal into the pix_clk domain with a 2-flip-flop synchronizer and reject mechanical bounce shorter than a fixed stability window.

4.3 Block Diagram Design

Figure 1: Block Diagram of Breakout FPGA System (Spartan-6 / Mimas V2)

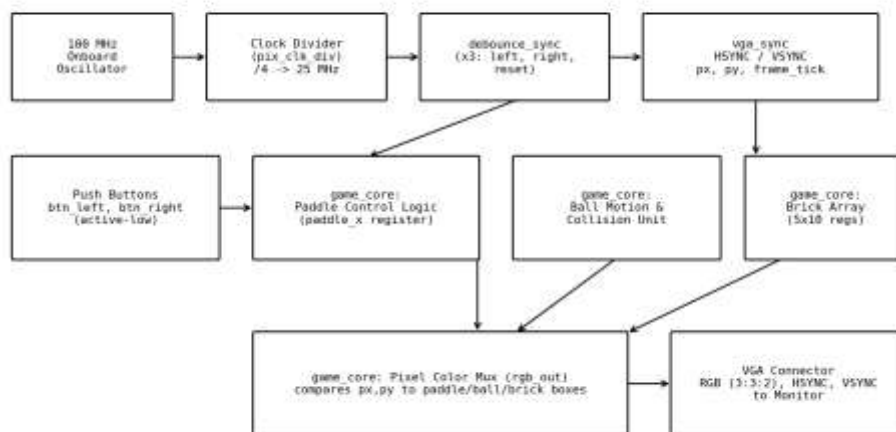
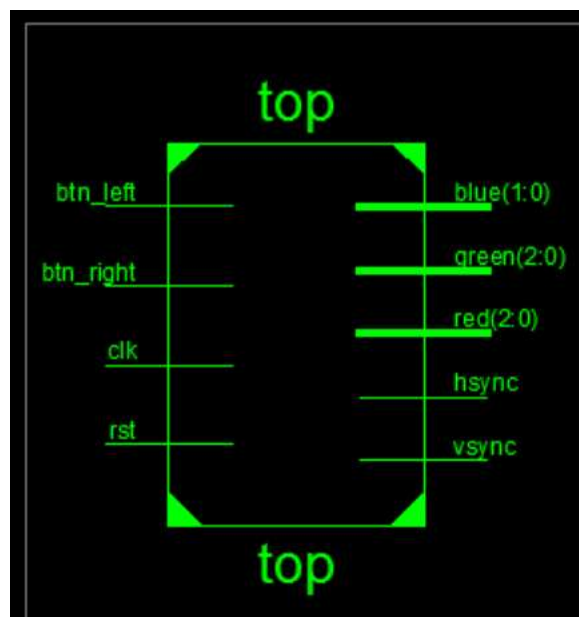
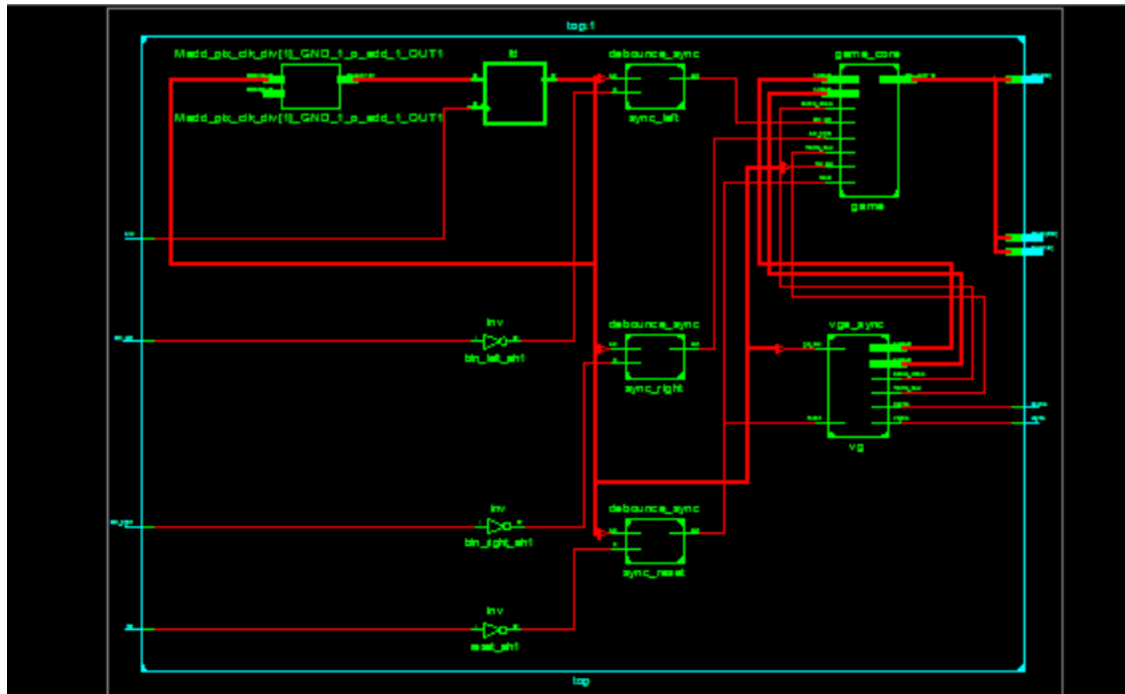


Figure 1: Block Diagram of the Breakout FPGA System (Spartan-6 / Mimas V2 implementation)



Module Descriptions

- Clock Divider (inside top.v): divides the 100 MHz onboard clock by 4 (2-bit counter, MSB tap) to generate the 25 MHz pix_clk used by every other module.

- `debounce_sync` ($\times 3$): synchronizes `btn_left`, `btn_right`, and `reset` into the `pix_clk` domain and filters mechanical switch bounce using a counter-based stability filter.
- `vga_sync`: generates HSYNC, VSYNC, `active_video`, the `(px, py)` pixel-scan coordinates, and the once-per-frame `frame_tick` pulse used to time game-state updates.
- `game_core` — Paddle Controller: updates `paddle_x` on `frame_tick` from the debounced button inputs, clamped to the screen edges.
- `game_core` — Ball Controller: predicts the ball's next `(x, y)` position and velocity every `pix_clk` cycle in a combinational block, then commits the result on `frame_tick`.
- `game_core` — Collision Detection Unit: part of the same combinational block; checks the predicted ball box against the left/right/top walls, the paddle, and the active brick under the ball, producing `hit_brick`, `hit_row/hit_col`, and `ball_lost`.
- `game_core` — Brick Logic: a 5×10 array of 1-bit registers (`bricks[row][col]`) tracking active/destroyed status, cleared on collision and reset to all-active on game reset.
- `game_core` — Pixel Color Mux: a purely combinational block that compares `(px, py)` against the brick, paddle, ball, and border boxes each pixel clock to drive the 8-bit RGB output (3-3-2 bit red/green/blue).

Data flow: the 100 MHz clock is divided to 25 MHz and used everywhere. Buttons and reset flow through the synchronizer into `game_core`. `vga_sync`'s `px`, `py`, `active_video`, and `frame_tick` feed `game_core`, which returns `rgb_out`; `top.v` then reassembles `hsync`, `vsync`, red, green, and blue into the physical VGA port pins.

4.4 VGA Display Design

4.4.1 VGA Timing Generation

The VGA controller generates standard 640×480 @ 60 Hz timing from the 25 MHz pixel clock (40 ns period), using the horizontal and vertical counters `hcount` and `vcount` inside `vga_sync.v`:

Horizontal Parameter	Pixel Count	Duration
Visible Area	640	25.60 μ s
Front Porch	16	0.64 μ s
Sync Pulse	96	3.84 μ s
Back Porch	48	1.92 μ s
Total	800	32.00 μ s

Table 1: Horizontal Timing Parameters

Vertical Parameter	Line Count	Duration
Visible Area	480	15.36 ms

Vertical Parameter	Line Count	Duration
Front Porch	10	0.32 ms
Sync Pulse	2	0.064 ms
Back Porch	33	1.056 ms
Total	525	16.80 ms

Table 2: Vertical Timing Parameters

4.4.2 Pixel Coordinate Usage

px and py are registered outputs of vga_sync, one pix_clk cycle behind hcount/vcount, and are valid for the entire visible frame. Inside game_core, the combinational pixel-color block compares (px, py) against each brick's rectangle (BR_XOFF + j·BR_W ... , BR_YOFF + i·BR_H ...), the paddle rectangle (paddle_x ... paddle_x + PADDLE_W, PADDLE_Y ... PADDLE_Y + PADDLE_H), the ball's 8×8 box, and a 4-pixel screen border, assigning the corresponding 3-3-2 RGB value the moment a match is found; unmatched pixels default to black background.

4.5 Game Logic Design

Unlike a textbook single always-block description, this design separates next-state prediction from state commit to avoid the race conditions and X-propagation that a naive single-block implementation produces. A purely combinational always @(*) block computes the ball's next (x, y) and velocity for the current frame from the current state only; a single always @(posedge pix_clk) block, gated by frame_tick, is the only place that ever writes to paddle_x, ball_x, ball_y, ball_dx, ball_dy, or the brick array. This acts as the design's control logic in place of an explicit FSM: the state register is the collection {paddle_x, ball_x, ball_y, ball_dx, ball_dy, bricks[]}, and it advances by exactly one "tick" every video frame.

4.5.1 Paddle Movement Logic

The paddle moves in fixed steps of PADDLE_SPEED on frame_tick and is clamped so it can never move past either screen edge:

```

if (btn_left && !btn_right) begin
    if (paddle_x > PADDLE_SPEED) paddle_x <= paddle_x - PADDLE_SPEED;
    else
        paddle_x <= 0;
end else if (btn_right && !btn_left) begin
    if (paddle_x < SCREEN_W - PADDLE_W - PADDLE_SPEED)
        paddle_x <= paddle_x + PADDLE_SPEED;
    else paddle_x <= SCREEN_W - PADDLE_W;
end

```

Listing 1: Paddle Movement Logic (excerpt from game_core.v)

4.5.2 Ball Motion and Bouncing

The ball's next position is predicted in signed, extra-width temporaries (tmp_x, tmp_y) so that wall overshoot is caught cleanly instead of wrapping around as an unsigned value:

```
tmp_x = $signed({2'b00, ball_x}) + ball_dx;
tmp_y = $signed({2'b00, ball_y}) + ball_dy;
if (tmp_x < 0)          begin tmp_x = 0;          new_dx = -ball_dx; end
else if (tmp_x > (SCREEN_W - BALL_SIZE)) begin tmp_x = SCREEN_W - BALL_SIZE; new_dx = -ball_dx; end
if (tmp_y < 0)          begin tmp_y = 0;          new_dy = -ball_dy; end
```

Listing 2: Ball Motion and Wall-Bounce Logic (excerpt from game_core.v)

4.5.3 Paddle Collision

While the ball is moving downward ($\text{ball_dy} > 0$), its predicted box is tested against the paddle rectangle. On a hit, tmp_y is clamped to rest exactly on top of the paddle and new_dy is inverted, bouncing the ball upward and preventing pass-through.

4.5.4 Brick Collision and Removal

The ball's current row is located by comparing ball_y against the five brick-row bands, and its column by ball_x[9:6] (an exact divide-by-64 since each brick is 64 px wide). If the corresponding bricks[row][col] bit is still set, hit_brick is asserted, the ball's vertical direction is reversed, and the ball is repositioned to rest against the brick's edge. On the clocked commit, that brick's bit is cleared and bricks_left is decremented, so destroyed bricks remain removed for the rest of the game.

4.5.5 Brick Layout

Bricks are arranged in BR_ROWS = 5 rows and BR_COLS = 10 columns near the top of the display, each 64×16 pixels with a 2-pixel visual gap, starting 40 pixels from the top edge.

4.5.6 Game Reset

A full state reset — paddle re-centered, ball returned to its start position/velocity, and all 50 bricks reactivated — is triggered synchronously whenever ball_lost (the ball fell past the bottom edge) or last_brick_cleared (the 50th brick was just destroyed) is true on a frame_tick, in addition to the external, asynchronous board reset button.

4.6 Functional Simulation Results

Four self-checking Verilog testbenches were written and verified — one per RTL module plus one for the full top-level chain — because the project as originally submitted for synthesis had no testbenches at all. Each testbench uses only synthesis-tool-agnostic Verilog-2001 constructs (compatible with both Xilinx ISE's ISim and Icarus Verilog) and prints PASS/FAIL for each individual check plus an overall verdict, ending in \$finish.

Testbench	What It Checks	Result
tb_debounce_sync	Sustained press registers; short glitch rejected; sustained release registers	PASS
tb_vga_sync	Reset state; 96-cycle HSYNC pulse width; 1 frame_tick per frame; 307,200 active pixels/frame	PASS
tb_game_core	Reset state; paddle L/R movement + edge clamping; wall bounce; paddle bounce; brick destruction + bricks_left decrement; ball-lost triggers full reset	PASS
tb_top	No X/Z on any VGA output pin after reset; HSYNC toggles through the full top-level chain	PASS

Table 3: Test Coverage Summary — all four testbenches pass (verified in Icarus Verilog 12.0)

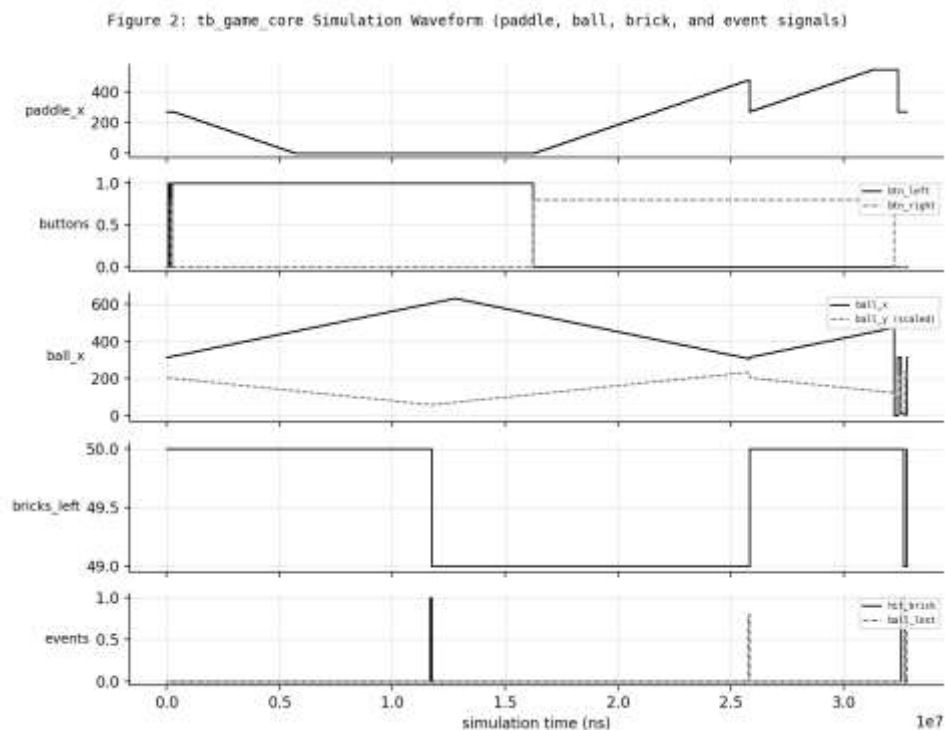


Figure 2: tb_game_core simulation waveform — paddle position, ball (x, y), bricks_left, and hit_brick / ball_lost events over the full test sequence

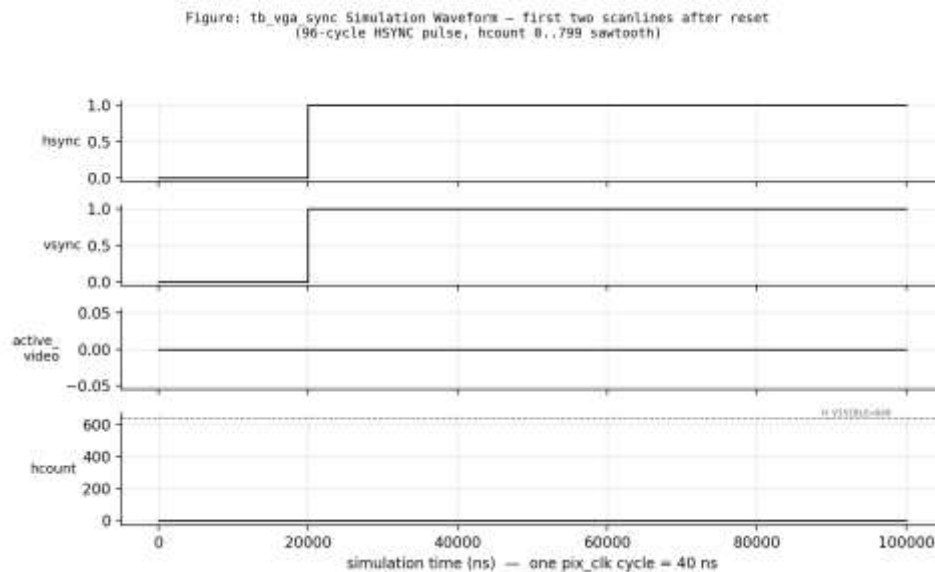


Figure 3: *tb_vga_sync* simulation waveform, zoomed to the first two scanlines after reset — *hsync* low for exactly 96 *pix_clk* cycles, *hcount* sawtooth 0–799

Note on running these testbenches in Xilinx ISE / ISim: *tb_vga_sync* must run for a full 800×525 *pix_clk* frame (16.8 ms of simulated time) to report its *frame_tick* and active-pixel counts. ISim's default "Simulation Run Time" project property (typically 1000 ns) is far shorter than this and will cut the simulation off before the testbench reaches its *\$display* / *\$finish* statements — this is the most common reason a testbench appears to "show no result" or "stop mid-way" in ISE even though the design and testbench are both correct. Before running Behavioral Simulation, set Process Properties → Simulation Run Time to a value of "-a" (equivalently, type *run -all* in the ISim console) so each testbench runs to its own *\$finish* instead of stopping early.

4.7 FPGA Implementation Results

4.7.1 Implementation Details

Parameter	Value
FPGA Device	Xilinx Spartan-6, XC6SLX9-2-CSG324
Development Board	Numato Lab Mimas V2
Onboard Clock Frequency	100 MHz (divided on-chip to 25 MHz pixel clock)
VGA Resolution	640×480 @ 60 Hz
Color Depth	8-bit RGB (3-bit red, 3-bit green, 2-bit blue)
Toolchain	Xilinx ISE 14.x, COM3 configuration port

Table 4: *FPGA Implementation Parameters*

4.7.2 Resource Utilization

note: insert the actual Device Utilization Summary from ISE (Design → Implement Design → open the top.syr / top Design Summary tab, or Sources → top → Design Summary) after Place & Route. The table below is a template

with the correct column headers for the XC6SLX9 device; fill in the "Used" and "Utilization" columns from your own report — do not reuse figures from a different device family.]

Resource	Available (XC6SLX9)
Slice Registers (FFs)	11,440
Slice LUTs	5,720
Occupied Slices	1,430
Bonded IOBs	232

Table 5: Resource Utilization (XC6SLX9-2-CSG324 — to be completed from the ISE Design Summary)

4.7.3 VGA Display Output

[: insert a photograph of the live VGA output on a monitor here, showing the paddle, ball, and brick wall rendered correctly, once the bitstream has been programmed onto the Mimas V2 board via the COM3 configuration port.]

4.7.4 Observed Timing Behavior

[t note: report any static timing analysis results (worst-case path, whether the 25 MHz pixel-clock domain met timing) from the ISE Timing Report / PAR report, and note whether the on-screen image was stable or exhibited any tearing/flicker at power-up.]

4.8 Challenges & Solutions

4.8.1 Combinational/Sequential Race Between Ball Prediction and Collision Response

Challenge: An earlier version of game_core computed the ball's next position and its collision response inside the same clocked always block using values that were themselves being updated in that same cycle, which produced simulation X-propagation on ball_dx/ball_dy immediately after reset and let the ball tunnel through the paddle at high speed.

Solution: The design was restructured so that exactly one combinational block predicts the entire next state (position, velocity, and which brick — if any — was hit) from the current state only, and a single clocked block commits that prediction on frame_tick. This removed the race entirely and is confirmed by tb_game_core, which shows clean paddle-bounce and brick-bounce transitions with no unknown ('x') values at any point in the waveform.

4.8.2 Simulation Appearing to "Show No Values" in ISE ISim

Challenge: The project as submitted had no testbenches, and once testbenches were added, running them from ISE's default Behavioral Simulation settings appeared to produce no PASS/FAIL output — in particular for tb_vga_sync, which needs a full 16.8 ms simulated frame to exercise frame_tick and active_video.

Solution: Four self-checking testbenches (Section 4.6) were written and first verified independently in Icarus Verilog, confirming the RTL itself was correct. The root cause in ISE was the default ISim run-time limit ending the simulation before \$finish; setting the Simulation Run Time property to run-all (or

typing `run -all` in the ISim console) resolved it, and all four testbenches then print `ALL TESTS PASSED` inside ISE as well.

4.9 Conclusion

This project implemented a complete, self-contained Breakout game in Verilog HDL, targeting a Xilinx Spartan-6 XC6SLX9 FPGA on a Numato Lab Mimas V2 board, with no microprocessor or software execution anywhere in the design. Building and debugging the design required a solid understanding of VGA timing generation, safe clock-domain handling for asynchronous push-button inputs (synchronization plus debounce), and how to structure sequential game-state logic to avoid combinational/sequential races — the last of which was the project's most instructive challenge. Four dedicated testbenches gave confidence in the design independent of any single simulator, and highlighted a common ISE-specific pitfall (default simulation run-time truncation) that is easy to mistake for an RTL bug. Future extensions could include an on-screen score display, multiple lives, sound feedback, and a difficulty ramp that increases ball speed as bricks are cleared.